Московский авиационный институт
(национальный исследовательский университет)

Факультет компьютерных наук и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №7 по курсу «Дискретный анализ»

Студент: Волков А.А.
Преподаватель: Михайлова С.А.
Группа: М8О-301БВ-24
Дата: 12 ноября 2026 г.
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №7

Задача: Дано натуральное число n. За один ход разрешается вычесть единицу, разделить число на два или на три. Деление допускается только нацело. Стоимость операции равна значению числа до её выполнения. Требуется с помощью динамического программирования найти минимальную стоимость преобразования n в единицу и вывести последовательность операций.

Формат ввода. Одно число $2 \le n \le 10^7$.

Формат вывода. В первой строке минимальная стоимость, во второй операции `-1`, `/2`, `/3`, разделённые пробелами. При наличии нескольких оптимальных последовательностей подходит любая.

Пример

Ввод	Вывод
82	202 -1 /3 /3 /3 /3

Стоимость пути $82 \to 81 \to 27 \to 9 \to 3 \to 1$ равна $82+81+27+9+3 = 202$. Конечная единица в сумму не входит.

1 Описание

Задачу можно представить как поиск кратчайшего пути в ориентированном графе. Вершины соответствуют числам от 1 до n. Из вершины i идут рёбра в $i-1$, а при делимости — также в $i/2$ и $i/3$. Вес каждого такого ребра равен i. Все рёбра ведут к меньшим числам.

Состояние и переходы

Вектор $dp[i]$ хранит минимальную стоимость пути **из исходного** n **в** i. Изначально $dp[n] = 0$, остальные значения равны $INF = 10^{15}$. Эта константа больше стоимости пути только через вычитание единицы: $n(n+1)/2 - 1 < 10^{14}$ при заданных ограничениях. Используется тип `long long`, поскольку промежуточные оценки могут превышать диапазон 32-битного целого числа.

Вершины обрабатываются в порядке $n, n-1, \ldots, 2$. Стоимость достижения следующего числа через i равна `cost = dp[i] + i`, поскольку операция стоит i.

```
dp[i - 1] = std::min(dp[i - 1], cost);

if (i % 2 == 0) {
    dp[i / 2] = std::min(dp[i / 2], cost);
}

if (i % 3 == 0) {
    dp[i / 3] = std::min(dp[i / 3], cost);
}
```

Каждое присваивание выбирает меньшую из уже найденной стоимости и стоимости пути через i. Таким образом обновляются ответы для $i-1$, $i/2$ и $i/3$.

Ответ находится в $dp[1]$. Все состояния достижимы через вычитания единицы, поэтому к моменту обработки вершины её стоимость конечна.

Восстановление последовательности

Дополнительные массивы предшественников и операций не используются. Начинаем с $x = 1$ и проверяем кандидатов $p = x + 1$, $p = 2x$, $p = 3x$, не превышающих n. Выбираем первый, удовлетворяющий равенству

$$dp[x] = dp[p] + p.$$

Запоминаем соответствующую операцию (`-1`, `/2`, `/3`) и переходим к p. Такой кандидат существует как последнее ребро кратчайшего пути. Индексы строго увеличиваются,

поэтому восстановление заканчивается в n. Собранный список разворачивается, чтобы операции шли от n к 1. Равенства вдоль найденного пути гарантируют, что его стоимость равна $dp[1]$.

Сложность

Инициализация вектора и обработка $n-1$ вершин требуют $O(n)$ времени. Если ответ содержит m операций, восстановление и разворот занимают $O(m)$, где $m \leq n-1$. Итоговая сложность — $O(n)$ по времени и памяти.

2 Исходный код

Листинг 1: main.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
#include <utility>

struct Result {
    long long cost;
    std::vector<std::string> operations;
};

Result solveDynamic(int n) {
    const long long INF = 1'000'000'000'000'000LL;
    std::vector<long long> dp(n + 1, INF);
    dp[n] = 0;

    for (int i = n; i > 1; --i) {
        const long long cost = dp[i] + i;

        dp[i - 1] = std::min(dp[i - 1], cost);

        if (i % 2 == 0) {
            dp[i / 2] = std::min(dp[i / 2], cost);
        }

        if (i % 3 == 0) {
            dp[i / 3] = std::min(dp[i / 3], cost);
        }
    }

    std::vector<std::string> operations;
```

```
32
33      int x = 1;
34      while (x < n) {
35          if (dp[x] == dp[x + 1] + (x + 1)) {
36              operations.emplace_back("-1");
37              ++x;
38          } else if (2 * x <= n && dp[x] == dp[2 * x] + 2 * x) {
39              operations.emplace_back("/2");
40              x *= 2;
41          } else if (3 * x <= n && dp[x] == dp[3 * x] + 3 * x) {
42              operations.emplace_back("/3");
43              x *= 3;
44          }
45      }
46
47      std::reverse(operations.begin(), operations.end());
48
49      return {dp[1], std::move(operations)};
50  }
51
52  int main() {
53      int n;
54      std::cin >> n;
55      const auto result = solveDynamic(n);
56      const auto& operations = result.operations;
57      std::cout << result.cost << '\n';
58
59      for (std::size_t i = 0; i < operations.size(); ++i) {
60          if (i > 0) {
61              std::cout << ' ';
62          }
63          std::cout << operations[i];
64      }
65      std::cout << '\n';
66
67      return 0;
68  }
```

3 Тест производительности

В дополнение к реализации через динамическое программирование взята реализация через рекурсию. Время работы представлено в таблице.

n	Динамическое программирование, с	Рекурсия, с
1000	0.000008311	131.072708593
10000	0.000085485	>180 (лимит)
100000	0.000441685	Переполнение стека
10000000	0.017758827	Переполнение стека

На завершившихся запусках динамический алгоритм в среднем быстрее рекурсивного в 15770991 раз. Незавершившиеся запуски в среднее ускорение не включены.

4 Выводы

В работе реализовано динамическое программирование для поиска минимальной стоимости преобразования числа в единицу. Представление задачи в виде ациклического графа позволило выбрать порядок обработки от n к 1: каждая вершина рассматривается после всех возможных предшественников.

Последовательность операций восстановлена по равенству стоимостей. При этом направление восстановления противоположно направлению выполнения операций, поэтому список разворачивается.

Итого: Разобрался с принципом решения задач динамического программированния. Сравнение с рекурсивным алгоритмом показало преимущество данного подхода, т.к. выполняется за линейное время и использует линейную память.

Список литературы

[1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — М.: Издательский дом «Вильямс», 2011. — 1296 с. Глава 15. Динамическое программирование.

[2] *Динамическое программирование — Викиконспекты ИТМО.*
URL: `neerc.ifmo.ru/wiki/` — «Динамическое программирование» (дата обращения: 12.11.2026).